

TB layers

Before you memorize UVM macros, you need a map of the testbench stack

Test, env, agent, and the DUT

- At the top sits the test, it builds the environment and kicks off sequences
- The environment holds agents and usually a scoreboard
- Each agent bundles a sequencer, a driver, and a monitor around one protocol
- The driver pushes transactions through a virtual interface into the DUT
- The scoreboard compares predict versus observe
- Two paths matter

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Testbench layer diagram

INTERACTIVE TOOL

Testbench layer diagram

Sketch the usual UVM **layer stack**: test → env → agent (sequencer / driver / monitor) + scoreboard → virtual interface → DUT. Starter: one active agent with the scoreboard on the analysis path.

Starter example: one active agent under the env, with a scoreboard on the monitor analysis path — click agent to see roles.

Load starter example

Challenges 0 / 22 cleared

Quiz: test: In this stack, the test typically...

☐ builds the env and starts the default sequence

☐ is the synthesizable DUT

☐ only dumps VCD files

☐ replaces the virtual interface

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

test → env

Test builds the env; env owns agents and analysis.

agent

Bundles sequencer, driver, and monitor for one protocol.

scoreboard

Compares predicted vs observed transactions.

vif → DUT

Virtual interface is the bridge to pins.

TB layers lab starter

learn_uvm2017 — tb layers

Real UVM literacy

```
esl - module01-tb-layers/examples/navigation

# Track A - real Unix
# real shell session (Track A - UVM layer literacy)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module01-tb-layers

$ ls examples/layer-sketch/
hierarchy.txt

$ cat examples/layer-sketch/hierarchy.txt
UVM testbench layer sketch (literacy - not runnable UVM)
=====

test (uvm_test)
├── env (uvm_env)
│   ├── agent (uvm_agent, active)
│   │   ├── sequencer → driver → virtual interface → DUT
│   │   └── monitor → analysis port → scoreboard
│   └── scoreboard (uvm_scoreboard)
└──

Stimulus path:
sequence → sequencer → driver → vif → DUT

Observe path:
DUT → vif → monitor → scoreboard (predict vs actual)

Notes:
- Passive agent: monitor only (no sequencer/driver activity).
- Env may hold multiple agents; scoreboard often shared.
- Flat classic TB collapses these roles into one module.

$ head -32 ../learn_uvm2017_sv_verilator/module6/examples/architecture/architecture.sv
/*
 * Module 6 Example 6.1: Testbench Architecture
 * Demonstrates a structured, hierarchical UVM environment skeleton.
 *
 * LEARNING OBJECTIVES:
 * 1. Understand UVM testbench hierarchy
 * 2. Learn component relationships (test → env → agent → driver/monitor/sequencer)
 * 3. Understand transaction flow (sequence → sequencer → drivers)
 * 4. Learn component creation and connection
 * 5. Understand UVM phases execution
 *
 * TESTBENCH HIERARCHY:
 * ArchitectureTest (uvm_test)
 * ├── ArchEnv (uvm_env)
 * │   └── ArchAgent (uvm_agent)
 * │       ├── ArchSequencer (uvm_sequencer)
 * │       ├── ArchDriver (uvm_driver)
 * │       └── ArchMonitor (uvm_monitor)
 *
 * TRANSACTION FLOW:
 * 1. Test creates sequence
 * 2. Sequence starts on sequencer
 * 3. Sequencer provides transactions to driver
 * 4. Driver drives DUT signals
 * 5. Monitor samples DUT signals (not implemented in this example)
 *
 * UVM PHASES:
 * - build_phase: Create components
 * - connect_phase: Connect components (driver-sequencer)
 * - run_phase: Execute test (sequences, driving, monitoring)
 * - report_phase: Generate reports
 */
```

Real shell — layer hierarchy sketch

Pitfalls to watch

- Do not confuse the test with the environment, the test is thin; the env owns structure
- Do not assume every agent is active, a passive agent monitors only and has no driver path
- An env without a scoreboard can still stimulate
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load the starter and finish a few challenges
- Restate the two paths, stimulus and observe, in your own words and skim one legacy example
- When you are ready